Comprehensive Theory of Single-Cycle Processor Design and the ToyRISC ISA

Part 1: Foundations of Computer Architecture

Computer Architecture Laboratory – Assignment 3

February 15, 2026

Contents

1 Introduction to Computer Architecture

Computer architecture is the science and art of designing the fundamental operational structure of a computer system. It encompasses the instruction set architecture (ISA), the microarchitecture, and the system design. Understanding computer architecture is essential for writing efficient software, designing hardware, and bridging the gap between high-level programming and the actual hardware that executes programs.

1.1 The Role of Computer Architecture

At its core, a computer is a machine that fetches instructions from memory, decodes them, executes them, and stores results. Computer architecture defines:

- **What operations the processor can perform** – defined by the Instruction Set Architecture (ISA).
- **How those operations are implemented in hardware** – defined by the microarchitecture.
- **How the processor interacts with memory and I/O** – defined by the system architecture.

1.2 Levels of Abstraction

Computer systems are organized in layers of abstraction, from high-level software to the transistors of the chip:

1. **Application Software**: Programs written in high-level languages (C, Java, Python).
2. **System Software**: Operating systems, compilers, assemblers that translate and manage.
3. **Instruction Set Architecture (ISA)**: The contract between hardware and software. Defines registers, instructions, memory model, and data types.
4. **Microarchitecture**: The implementation of the ISA in hardware. Includes pipeline design, cache hierarchy, branch prediction, etc.
5. **Register-Transfer Level (RTL)**: The description of data flow between registers and the combinational logic that transforms data.
6. **Gate Level**: Logic gates (AND, OR, NOT, XOR, MUX) that implement the RTL.
7. **Transistor Level**: Physical CMOS transistors that implement the gates.

1.3 Von Neumann Architecture

The Von Neumann architecture, proposed by John von Neumann in 1945, is the foundation of nearly all modern computers. Its key principles are:

1. **Stored Program Concept**: Both instructions and data reside in the same memory. This allows programs to be modified, loaded dynamically, and treated as data.

2. **Sequential Execution**: Instructions are fetched and executed one at a time, in the order they appear in memory (unless a branch or jump alters control flow).
3. **Single Memory Space**: A unified address space holds both code and data (though the ToyRISC ISA separates them logically into segments).
4. **Central Processing Unit (CPU)**: Contains the Arithmetic Logic Unit (ALU) for computation and the Control Unit (CU) for sequencing operations.

The **Von Neumann bottleneck** is the limitation that arises because the CPU can only fetch one instruction or one data value per memory access cycle. This bottleneck motivates caches, pipelining, and Harvard architecture variants.

1.4 Harvard Architecture

The Harvard architecture separates instruction memory from data memory, allowing simultaneous access to both. Many modern processors use a **modified Harvard architecture**, which has separate instruction and data caches (L1i and L1d) but a unified main memory. The ToyRISC simulator's configuration file includes parameters for both L1i and L1d caches, reflecting this design philosophy.

2 Instruction Set Architecture (ISA) – General Theory

The ISA is the most important abstraction in computer architecture. It defines the interface between software and hardware.

2.1 What an ISA Defines

An ISA specifies the following:

- **Data Types**: Integer (signed/unsigned), floating point, character, boolean. ToyRISC uses 32-bit signed integers.
- **Registers**: The number, size, and purpose of registers. ToyRISC has 32 general-purpose registers (`x0`–`x31`), each 32 bits wide.
- **Instruction Formats**: The binary encoding of instructions. ToyRISC uses 32-bit fixed-width instructions in three formats (R3-Type, R2I-Type, RI-Type).
- **Instruction Types**: Arithmetic, logical, memory access, control flow, and special instructions.
- **Addressing Modes**: How operand addresses are computed. ToyRISC supports register, immediate, and register+immediate (base+offset) addressing.
- **Memory Model**: Address space size, byte ordering, alignment. ToyRISC has a 16-bit address space ($2^{16} = 65536$ words).
- **Exception and Interrupt Handling**: How the processor responds to errors and external events (not covered in ToyRISC).

2.2 RISC vs. CISC

2.2.1 CISC (Complex Instruction Set Computer)

CISC architectures (e.g., x86) have:

- Variable-length instructions (1 to 15 bytes in x86).
- Complex instructions that can perform multiple operations (e.g., REP MOVSB moves a block of memory).
- Many addressing modes (register, immediate, direct, indirect, indexed, based+indexed, etc.).
- Fewer registers (historically 8 in x86).
- Microprogrammed control units.

2.2.2 RISC (Reduced Instruction Set Computer)

RISC architectures (e.g., MIPS, ARM, RISC-V, and ToyRISC) have:

- Fixed-length instructions (32 bits in ToyRISC).
- Simple instructions that each perform one operation.
- Load-store architecture: only `load` and `store` access memory; all other operations work on registers.
- Many general-purpose registers (32 in ToyRISC, same as RISC-V and MIPS).
- Hardwired control units.
- Regular instruction formats that simplify decoding.

ToyRISC is a classic RISC architecture. It follows the load-store paradigm, has fixed 32-bit instructions, and uses a simple, regular encoding scheme.

2.3 Instruction Encoding Fundamentals

Every instruction must be encoded as a binary number. The encoding must contain enough information for the processor to determine:

1. **What operation to perform** (opcode).
2. **What data to operate on** (operands: register numbers, immediate values).
3. **Where to put the result** (destination register or memory address).

In a fixed-width ISA like ToyRISC (32 bits per instruction), the challenge is to pack all this information into a fixed number of bits. This requires trade-offs:

- More bits for the opcode ⇒ more distinct operations but fewer bits for operands.
- More bits for immediate values ⇒ larger constants but fewer bits for register specifiers.
- More register specifier fields ⇒ more operands but smaller immediates.

2.4 Addressing Modes

Addressing modes define how the processor computes the effective address of an operand:

Mode	Example	Description
Register	`add x3, x4, x5`	Operand is in a register
Immediate	`addi x3, 5, x5`	Operand is a constant embedded in the instruction
Base + Offset	`load x3, 10, x5`	Memory address = register value + immediate
PC-Relative	`beq x3, x4, label`	Branch target = PC + offset

ToyRISC uses all four of these modes. The base+offset mode is used by `load` and `store`. PC-relative mode is used by conditional branches (`beq`, `bne`, `blt`, `bgt`) and unconditional jumps (`jmp`).

3 Registers and the Register File

3.1 What are Registers?

Registers are the fastest storage elements in a processor. They are implemented using flip-flops (latches) within the CPU itself, so accessing a register takes zero additional clock cycles beyond the pipeline stage that reads them. By contrast, main memory access can take tens to hundreds of clock cycles.

3.2 The ToyRISC Register File

ToyRISC has **32 general-purpose integer registers**, each 32 bits wide, denoted `x0` through `x31`. Additionally, there is a **Program Counter (PC)** register that holds the address of the current instruction.

3.2.1 Special Registers

Register	Name	Purpose
`x0`	Zero Register	Always contains the value 0. Cannot be overwritten. Used as a source of zero for operations like clearing a register (`add x0, x0, x3` sets `x3 = 0`).
`x1`	Stack Pointer (SP)	Points to the top of the stack. Initialized to $2^{16} - 1 = 65535$. The stack grows downward (toward lower addresses).
`x2`	Frame Pointer (FP)	Points to the base of the current stack frame. Initialized to $2^{16} - 1 = 65535$. Used to access function arguments and local variables at fixed offsets.

`x31`	Overflow / Remainder	Stores the upper 32 bits of a 64-bit multiplication result, the remainder of a division, or the bits shifted out during shift operations.
PC	Program Counter	Holds the memory address of the instruction currently being fetched. Incremented by 1 after each fetch (since each instruction occupies one memory word).

3.3 Register File Implementation

In the Java simulator, the register file is implemented by the `RegisterFile.java` class:

```java
public class RegisterFile {
    int[] registerFile;  // 32 registers, each 32-bit int
    int programCounter;  // PC register

    public RegisterFile() {
        registerFile = new int[32];
        registerFile[0] = 0;  // x0 is always 0
    }

    public int getValue(int registerNumber) {
        return registerFile[registerNumber];
    }

    public void setValue(int registerNumber, int value) {
        registerFile[registerNumber] = value;
    }

    public int getProgramCounter() {
        return programCounter;
    }

    public void setProgramCounter(int programCounter) {
        this.programCounter = programCounter;
    }

    public void incrementProgramCounter() {
        this.programCounter++;
    }
}
```

Listing 1: RegisterFile.java – Simplified

Key observations:

- The register file is a simple array of 32 integers.
- The PC is stored as a separate integer field.
- Register numbering requires 5 bits ($\log_2 32 = 5$), which is why each register specifier in the instruction format is 5 bits.
- In a real hardware implementation, the register file would have multiple read ports (typically 2) and one write port, allowing the processor to read two source registers and write one destination register simultaneously.

4 Memory System

4.1 Main Memory in ToyRISC

The ToyRISC processor has a **word-addressable** main memory with $2^{16} = 65536$ words. Each word is 32 bits (an `int` in Java). This means the total memory capacity is:

$$65536 \times 32 \text{ bits} = 65536 \times 4 \text{ bytes} = 262144 \text{ bytes} = 256 \text{ KB}$$

Note that ToyRISC uses **word addressing**, not byte addressing. Address 0 refers to the first 32-bit word, address 1 refers to the second 32-bit word, and so on. This is different from most real architectures (x86, ARM, RISC-V) which use byte addressing.

4.2 Address Space Layout

The ToyRISC address space is divided into three segments:

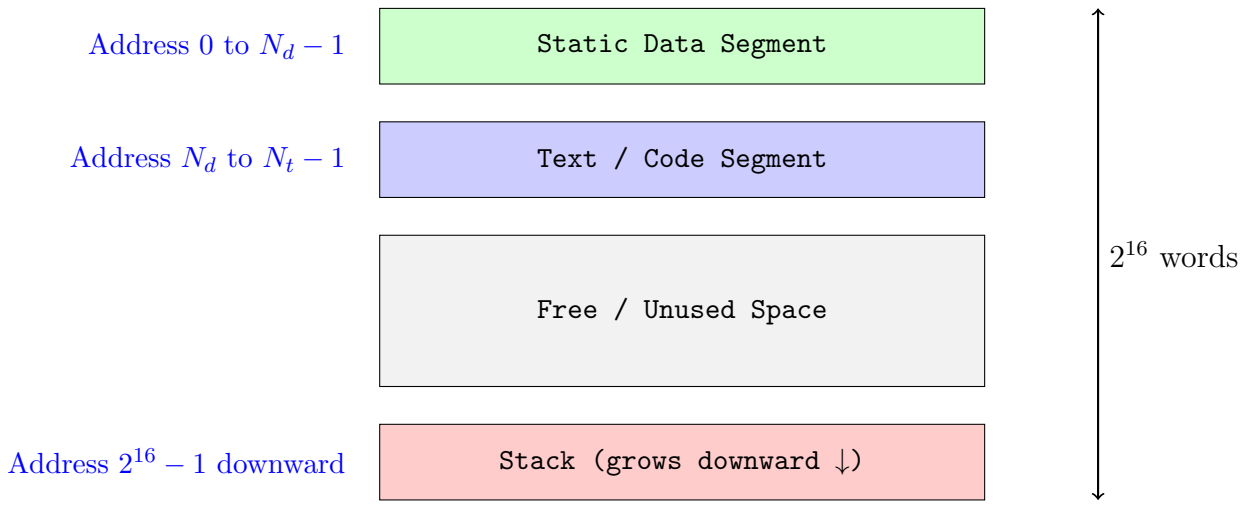

Figure 1: ToyRISC Address Space Layout

4.2.1 Static Data Segment (Address 0 to $N_d - 1$)

The static data segment starts at address 0 and contains global variables and constants declared in the `.data` section of the assembly program. For example:

```
.data
a:
    123
    234
```

Here, value 123 is stored at address 0, and value 234 is stored at address 1. The label `a` is an alias for address 0.

4.2.2 Text / Code Segment (Address N_d to $N_t - 1$)

The text segment immediately follows the data segment. It contains the machine code instructions of the program. Each instruction occupies exactly one 32-bit word (one

memory address). If the data segment contains N_d words, then the first instruction is at address N_d.

4.2.3 Stack Segment (Address $2^{16} - 1$ downward)

The stack starts at the highest address ($2^{16} - 1 = 65535$) and grows **downward** (toward lower addresses). The stack pointer (`x1`) and frame pointer (`x2`) are both initialized to 65535.

Push operation: Decrement the stack pointer by 1, then store the value at the address pointed to by the stack pointer.

Pop operation: Load the value from the address pointed to by the stack pointer, then increment the stack pointer by 1.

4.3 Memory Implementation

```java
public class MainMemory {
    int[] memory;

    public MainMemory() {
        memory = new int[65536];  // 2^16 words
    }

    public int getWord(int address) {
        return memory[address];
    }

    public void setWord(int address, int value) {
        memory[address] = value;
    }
}
```

Listing 2: MainMemory.java

4.4 Memory Hierarchy (Preview)

While the single-cycle simulator in Lab 3 does not implement caches, the configuration file (`config.xml`) defines a memory hierarchy for future assignments:

1. **L1 Instruction Cache (L1i)**: 256 lines, 2-cycle latency, 4-way set-associative, LRU replacement.
2. **L1 Data Cache (L1d)**: 256 lines, 2-cycle latency, 4-way set-associative, LRU replacement.
3. **L2 Unified Cache**: 2048 lines, 10-cycle latency, 4-way set-associative, LRU replacement.
4. **Main Memory**: 40-cycle latency.

This hierarchy illustrates the principle of **locality of reference**: recently accessed data (temporal locality) and nearby data (spatial locality) are likely to be accessed again soon, so keeping them in fast, small caches dramatically improves performance.

5 The Single-Cycle Processor Model

5.1 What is a Single-Cycle Processor?

A single-cycle processor executes **one instruction per clock cycle** (or equivalently, one instruction passes through all stages before the next begins). In this model:

- The clock period must be long enough to accommodate the **slowest instruction** (typically a load instruction, which requires memory access).
- All instructions take the same amount of time, even simple ones like `add` that don't need memory access.
- There is no overlap between instructions – no pipelining.

5.2 Stages of Instruction Execution

Every instruction, regardless of type, passes through the same sequence of stages:

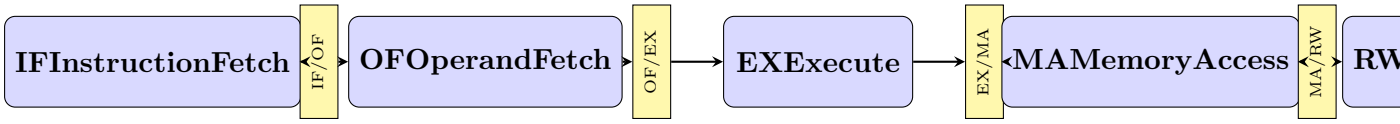

Figure 2: Five-Stage Single-Cycle Pipeline with Inter-Stage Latches

5.2.1 Stage 1: Instruction Fetch (IF)

Purpose: Retrieve the next instruction from memory.

Actions:

1. Read the Program Counter (PC) to get the address of the current instruction.
2. Access main memory at that address to fetch the 32-bit instruction word.
3. Increment the PC by 1 (pointing to the next sequential instruction).
4. Write the fetched instruction into the IF/OF latch.

Hardware components used: Program Counter register, instruction memory (or instruction cache), adder (for PC+1).

Java implementation (provided in the skeleton):

```
public void performIF() {
    if(IF_EnableLatch.isIF_enable()) {
        int currentPC = containingProcessor.getRegisterFile()
                        .getProgramCounter();
        int newInstruction = containingProcessor.getMainMemory()
                             .getWord(currentPC);
        IF_OF_Latch.setInstruction(newInstruction);
        containingProcessor.getRegisterFile()
                           .setProgramCounter(currentPC + 1);

        IF_EnableLatch.setIF_enable(false);
```

```
        IF_OF_Latch.setOF_enable(true);
    }
}
```

Listing 3: InstructionFetch.java – performIF()

5.2.2 Stage 2: Operand Fetch / Instruction Decode (OF/ID)

Purpose: Decode the instruction and read the required register values.
Actions:

1. Read the 32-bit instruction word from the IF/OF latch.
2. Extract the opcode (bits 27–31) to determine the operation type.
3. Based on the instruction format (R3-Type, R2I-Type, or RI-Type), extract:
   - Source register 1 (`rs1`) – bits 22–26
   - Source register 2 (`rs2`) or immediate – depends on format
   - Destination register (`rd`) – depends on format
4. Read the values of the source registers from the register file.
5. Sign-extend the immediate value (if applicable).
6. Write all decoded information into the OF/EX latch.

Hardware components used: Instruction decoder (combinational logic), register file (read ports), sign extender, multiplexers.

5.2.3 Stage 3: Execute (EX)

Purpose: Perform the computation specified by the instruction.
Actions (depend on instruction type):

- **Arithmetic/Logical**: Compute the result using the ALU (e.g., add, subtract, multiply, divide, AND, OR, XOR, SLT, shifts).
- **Memory**: Compute the effective address: base register + immediate offset.
- **Branch**: Compare the two source registers and determine whether the branch is taken. If taken, compute the target address: PC + immediate offset.
- **Jump**: Compute the target address: PC + rd + immediate.

Hardware components used: Arithmetic Logic Unit (ALU), comparator, adder, barrel shifter, multiplier, divider.

5.2.4 Stage 4: Memory Access (MA)

Purpose: Access data memory for load and store instructions.

Actions:

- **Load**: Read the data word from main memory at the effective address computed in the EX stage.
- **Store**: Write the data word to main memory at the effective address.
- **Other instructions**: This stage is a pass-through (no memory operation needed).

Hardware components used: Data memory (or data cache), address bus, data bus.

5.2.5 Stage 5: Register Write-Back (RW)

Purpose: Write the result of the instruction back to the register file.

Actions:

- **Arithmetic/Logical**: Write the ALU result to the destination register.
- **Load**: Write the loaded data to the destination register.
- **Store**: No register write needed.
- **Branch/Jump**: No register write needed (but PC may have been updated in EX stage).
- **End**: Signal simulation completion.

Hardware components used: Register file (write port), multiplexer to select between ALU result and memory data.

5.3 The Clock and Timing

In a single-cycle processor:

$$T_{\text{clock}} \geq T_{\text{IF}} + T_{\text{OF}} + T_{\text{EX}} + T_{\text{MA}} + T_{\text{RW}}$$

The clock period must be long enough for the signal to propagate through **all five stages** in a single cycle. This is determined by the critical path, which is typically the `load` instruction (it uses all five stages actively).

For ToyRISC in the simulator, the clock is simply a counter:

```java
public class Clock {
    static long currentTime = 0;

    public static void incrementClock() {
        currentTime++;
    }

    public static long getCurrentTime() {
        return currentTime;
    }
}
```

Listing 4: Clock.java

In the `simulate()` function, the clock increments after each stage:

```
public static void simulate() {
    while(simulationComplete == false) {
        processor.getIFUnit().performIF();
        Clock.incrementClock();
        processor.getOFUnit().performOF();
        Clock.incrementClock();
        processor.getEXUnit().performEX();
        Clock.incrementClock();
        processor.getMAUnit().performMA();
        Clock.incrementClock();
        processor.getRWUnit().performRW();
        Clock.incrementClock();
    }
}
```

Listing 5: Simulator.java – simulate() loop

Each instruction takes **5 clock ticks** (one per stage). This is the single-cycle model: one complete instruction per 5-tick "cycle."

5.4 Performance Metrics

For a single-cycle processor:

- **CPI (Cycles Per Instruction)** = 1 instruction per "instruction cycle" (but each instruction cycle is 5 clock ticks in the simulator).
- **Execution Time** = Number of Instructions × CPI × Clock Period
- **Throughput** = $\frac{1}{\text{Clock Period}}$ instructions per second
- **Latency** = Time to complete one instruction = Clock Period

The number of cycles reported by the simulator is 5× (number of instructions), since the clock increments 5 times per instruction.

5.5 Advantages and Disadvantages

Advantages of Single-Cycle Design

- **Simplicity**: No hazard detection, no forwarding, no stalling.
- **Correctness**: Easy to verify because there's no overlap between instructions.
- **Deterministic**: Every instruction takes exactly the same time.
- **No pipeline hazards**: Data hazards, control hazards, and structural hazards don't exist.

Disadvantages of Single-Cycle Design

- **Slow**: The clock period is determined by the slowest instruction. Simple instructions (like `add`) are unnecessarily slow.
- **Poor resource utilization**: Most hardware components are idle for most of the clock cycle. The ALU is only active during the EX stage; memory is only active during IF and MA.
- **Not scalable**: As the ISA grows more complex, the critical path lengthens, and the clock period must increase.

6 Inter-Stage Latches

6.1 Purpose of Latches

In a processor pipeline (even a single-cycle one simulated sequentially), **latches** (also called **pipeline registers**) are buffers that sit between two consecutive stages. They serve two purposes:

1. **Communication**: They pass the output of one stage to the input of the next stage.
2. **Synchronization**: In a pipelined processor, they ensure that each stage works on a different instruction simultaneously. In a single-cycle processor, they simply hold intermediate results.

6.2 ToyRISC Latch Types

The ToyRISC simulator has the following latches:

Latch	Between Stages	Contents
`IF_EnableLatchType`	Enable → IF	Boolean: whether IF stage should execute
`IF_OF_LatchType`	IF → OF	The 32-bit instruction word; OF enable flag
`OF_EX_LatchType`	OF → EX	Decoded instruction (opcode, source operand values, destination register, immediate), EX enable flag
`EX_MA_LatchType`	EX → MA	ALU result, effective address, data to store, MA enable flag
`EX_IF_LatchType`	EX → IF	Branch/jump target PC (for control flow instructions)
`MA_RW_LatchType`	MA → RW	ALU result or loaded data, destination register, RW enable flag

Note the `EX_IF_LatchType`: this is a **feedback latch** from the Execute stage back to the Instruction Fetch stage. It is used when a branch or jump instruction needs to redirect the PC.

6.3 Latch as a Handshake Mechanism

Each latch has an "enable" flag that implements a simple handshake:

1. Stage N completes its work and writes results to the $N/N{+}1$ latch.
2. Stage N sets the enable flag of Stage $N{+}1$ to `true`.
3. Stage $N{+}1$ checks its enable flag; if `true`, it proceeds.
4. Stage $N{+}1$ sets its own enable flag to `false` after reading.

For example, in `InstructionFetch.performIF()`:

```
IF_EnableLatch.setIF_enable(false);  // disable IF
IF_OF_Latch.setOF_enable(true);     // enable OF
```

7 The Instruction Lifecycle – Complete Walkthrough

Let us trace the complete lifecycle of an instruction through all five stages using a concrete example:

Example: `add %x3, %x4, %x5` (add values in registers x3 and x4, store result in x5)
Assume: `x3 = 10`, `x4 = 20`, PC = 5 (instruction at memory address 5).

7.1 Step 1: Instruction Fetch (IF)

1. Read PC = 5.
2. Fetch instruction from `memory[5]`. This returns the 32-bit binary encoding of `add %x3, %x4, %x5`.
3. Increment PC to 6.
4. Write the 32-bit instruction to the IF/OF latch.
5. Set OF enable = true, IF enable = false.

7.2 Step 2: Operand Fetch (OF)

1. Read the 32-bit instruction from the IF/OF latch.
2. Extract opcode = 00000 (add).
3. This is R3-Type, so extract: rs1 = x3 (bits 22–26), rs2 = x4 (bits 17–21), rd = x5 (bits 12–16).
4. Read register file: `x3` = 10, `x4` = 20.

5. Write to OF/EX latch: opcode = add, operand1 = 10, operand2 = 20, destination = x5.
6. Set EX enable = true, OF enable = false.

7.3 Step 3: Execute (EX)

1. Read from OF/EX latch: opcode = add, operand1 = 10, operand2 = 20.
2. Compute: result = 10 + 20 = 30.
3. Write to EX/MA latch: result = 30, destination = x5.
4. Set MA enable = true, EX enable = false.

7.4 Step 4: Memory Access (MA)

1. Read from EX/MA latch: result = 30, instruction type = add.
2. Since `add` is not a memory instruction, this is a pass-through.
3. Write to MA/RW latch: result = 30, destination = x5.
4. Set RW enable = true, MA enable = false.

7.5 Step 5: Register Write (RW)

1. Read from MA/RW latch: result = 30, destination = x5.
2. Write 30 to register x5.
3. Set IF enable = true (allowing the next instruction to be fetched).
4. Set RW enable = false.

Result: Register x5 now contains 30. PC = 6 (ready to fetch next instruction).

8 Number Representation and Binary Arithmetic

8.1 Two's Complement Representation

ToyRISC uses 32-bit signed integers in two's complement. In two's complement:

- The most significant bit (bit 31) is the **sign bit**: 0 for positive, 1 for negative.
- Range: -2^{31} to $2^{31} - 1$ (i.e., $-2,147,483,648$ to $2,147,483,647$).
- To negate a number: invert all bits and add 1.
- Addition and subtraction use the same hardware circuit, which is why two's complement is universal in computer architecture.

8.2 Sign Extension

When an immediate value with fewer bits (e.g., 17 bits in R2I-Type or 22 bits in RI-Type) needs to be used in a 32-bit operation, it must be **sign-extended**: the sign bit is replicated to fill the upper bits.

For example, a 17-bit immediate value of -5:

- 17-bit: `1 1111 1111 1111 1011`
- 32-bit (sign-extended): `1111 1111 1111 1111 1111 1111 1111 1011`

8.3 Overflow and Special Handling in ToyRISC

According to the ISA specification:

- **Multiplication**: If the 64-bit result exceeds 32 bits, the upper 32 bits (bits 63–32) are stored in `x31`.
- **Division**: The quotient goes to the destination register, and the **remainder** goes to `x31`.
- **Shifts**: The bits shifted out are stored in `x31`.

8.4 Bitwise Operations

ToyRISC supports the standard bitwise operations:

Operation	Symbol	Description
AND	`&`	Bitwise AND. Result bit is 1 only if both input bits are 1.
OR	`|`	Bitwise OR. Result bit is 1 if either input bit is 1.
XOR	`^`	Bitwise XOR. Result bit is 1 if inputs differ.
SLL	$\ll$	Shift Left Logical. Shifts bits left, fills right with zeros.
SRL	$\gg$ (logical)	Shift Right Logical. Shifts bits right, fills left with zeros.
SRA	$\gg$ (arith.)	Shift Right Arithmetic. Shifts bits right, fills left with sign bit (preserves sign).

9 Control Flow and Branching

9.1 Sequential Execution

By default, instructions execute sequentially: after executing the instruction at address A, the next instruction fetched is at address $A + 1$. This is achieved by incrementing the PC in the IF stage.

9.2 Unconditional Jump (jmp)

The `jmp` instruction changes the PC unconditionally:

$$\text{PC} = \text{PC} + \text{rd} + \text{imm}$$

In assembly, either `rd` or `imm` is used (the other is set to zero). For example:

```
jmp loop        ; Jump to the label 'loop'
```

In machine code, the label is converted to a PC-relative offset. If the current PC is 10 and the label `loop` is at address 7, then the offset would be $7 - 10 = -3$ (since $\text{PC}_{\text{new}} = \text{PC} + \text{offset}$). However, note that the PC has already been incremented by 1 during the IF stage, so the actual offset stored needs to account for this.

9.3 Conditional Branches

ToyRISC has four conditional branch instructions:

Instruction	Condition	Action
`beq`	rs1 == rd	If true, PC = PC + imm
`bne`	rs1 $\neq$ rd	If true, PC = PC + imm
`blt`	rs1 ¡ rd	If true, PC = PC + imm
`bgt`	rs1 ¿ rd	If true, PC = PC + imm

Important: In the `ParsedProgram` (object file) representation:

- The two registers being compared are placed in `sourceOperand1` and `sourceOperand2`.
- The immediate offset is placed in `destinationOperand`.

This is a critical implementation detail for the Operand Fetch stage.

9.4 Branch Target Calculation

For branches, the target address is computed **relative to the current PC**:

$$\text{Target} = \text{PC}_{\text{at branch}} + \text{imm}$$

Since PC has already been incremented during IF, the actual target computed in EX is:

$$\text{Target} = \text{PC}_{\text{original}} + \text{imm}$$

The EX stage communicates the new PC back to the IF stage via the `EX_IF_LatchType`.

10 Function Calling Convention

The ToyRISC ISA defines a standard calling convention for function calls, specifying how arguments are passed, how the stack is managed, and how control is transferred.

10.1 Stack Operations

Push (storing a value onto the stack):

1. Decrement x1 (stack pointer) by 1.
2. Store the value at the address pointed to by x1.

Pop (retrieving a value from the stack):

1. Load the value from the address pointed to by x1.
2. Increment x1 by 1.

10.2 Caller Responsibilities

Before calling a function:

1. **Save caller-saved registers**: Push any registers whose values the caller needs preserved.
2. **Push arguments**: Push function arguments onto the stack.
3. **Push return address**: Push the address of the instruction following the `jmp` to the function.
4. **Jump to function**: Execute `jmp` to the function's entry point.

After the function returns:

1. Read return values from addresses below x1.
2. Pop the previously saved registers.

10.3 Callee Responsibilities

Upon entering the function:

1. **Save x2**: Push the old frame pointer (x2) onto the stack.
2. **Set up frame**: x2 = x1 - 1 (frame pointer points to the base of the current frame).
3. **Perform computation**: Access arguments via x2 + offsets. Use stack space below x2 for local variables.

Upon exiting the function:

1. **Restore x1**: x1 = x2 (collapse the stack frame).
2. **Restore x2**: Pop the saved frame pointer into x2.
3. **Push return values**: Push return values onto the stack.
4. **Return**: Load the return address from the stack and jump to it.

10.4 Stack Frame Layout

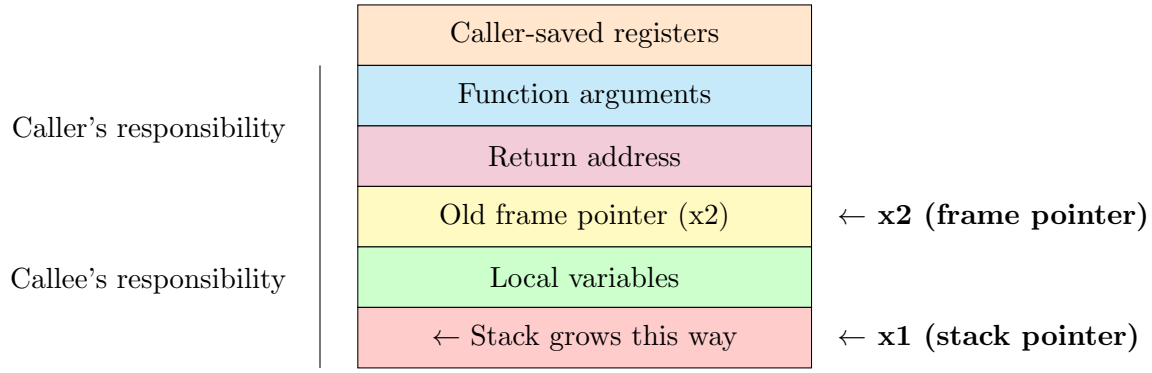

Figure 3: Stack Frame Layout in ToyRISC

11 Comparison with RISC-V

ToyRISC is intentionally designed to resemble RISC-V. Here is a comparison:

Feature	**ToyRISC**	**RISC-V (RV32I)**
Word size	32 bits	32 bits
Registers	32 (x0–x31)	32 (x0–x31)
x0 behavior	Always 0	Always 0
Instruction length	Fixed 32 bits	Fixed 32 bits (base)
Formats	3 (R3, R2I, RI)	6 (R, I, S, B, U, J)
Opcode bits	5	7
Addressing	Word-addressable	Byte-addressable
Memory size	2^{16} words	2^{32} bytes
Endianness	N/A (word)	Little-endian

End of Part 1. Continue to Part 2 for detailed ToyRISC ISA specification, instruction encoding, and pipeline stage implementation.